

Managing accounts

Group accounts, sharing one, changing an account afterwards, and deleting it.

This page continues Account types.

Group accounts

A group account is a shared workspace with **no Windows account and no sign-in of its own**. It is how you keep separate work separate, and it is the one kind of account that needs no way in from outside — which makes it the only extra account available on a machine with no ssh server and no API.

```
create.account group payroll  
modify.account add payroll fred
```

Reach it with `logto payroll`, or through an F pointer. On a machine with no remote access at all, `logto` into a group account from a session run as administrator.

Sharing a user account

```
grant <account> to <user>  
revoke <account> from <user>  
list.grants <account>
```

All three need an elevated session, and say so rather than failing obscurely.

Read this twice, because it is the most confusing part of how SD controls access. Entry to an account is membership of the Windows group named in the account's record, and **Windows fixes group membership when you sign in**. So:

- a grant does not reach the person until they sign out of Windows and back in;
- and **somebody you have just revoked keeps the account until they do the same**.

Both verbs print that reminder every time.

`list.grants` also shows the account's own user, which is always there and is not a grant. It is listed anyway so that what you see matches the Windows group it is reporting.

The old field 4 of an `ACCOUNTS` record — a list of accounts allowed in — has been removed, and `list accounts` no longer shows a *Granted to* column. `list.grants` answers that question now.

`list accounts` **shows the tier, and its columns have changed.** The default is `Account`, `Pathname`, `Description`, `Tier`. The Windows group came out to make room: it is always `sdu_` or `sdg_` followed by the account name, so it told you nothing the id did not. Both are still there by name — `list accounts path descr group tier`. `Was` **is the tier a suspension displaced**, and it is empty on an account that is not suspended.

From an ordinary account the register is reached as `sd.accounts`, not `accounts` — `list sd.accounts`, `ct sd.accounts fred`.

Changing an account afterwards

```
modify.account <name> <standard | programmer | administrator | suspended>
modify.account <name> <ssh | api | both | none>
modify.account <name> <sh-on | sh-off | os-on | os-off>
modify.account add <group> <user>
modify.password {<account>}
```

All of them need an elevated session.

THE TIER

The tier moves in either direction and needs no intermediate step. An account can go programmer, standard, suspended and back to any of them directly. Naming a tier on a suspended account lifts the suspension into that tier — **there is no `resume` keyword**; coming back always says where to.

The VOC is rebuilt there and then, not at the next login, and the verb says what it did:

```
:modify.account fred standard
Account FRED is now STANDARD
VOC: 0 records added, 42 removed, 0 left alone
```

"Left alone" is the count to read. A record is only removed if it is still exactly what SD put there. **Anything you or the account's owner has edited is counted and kept** — it is somebody's work under a verb's name rather than the verb, and a downgrade will not destroy it.

LEAVING `ADMINISTRATOR` TAKES THREE THINGS, AND YOU MUST NAME ONE

Windows <code>Administrators</code> membership	removed for you
--	-----------------

the <code>os.users</code> record	removed for you
----------------------------------	-----------------

ssh and the API	you say
-----------------	----------------

The first two were the account's *because* it was an administrator. ssh and the API are a rule for an administrator and a setting for everybody else, so the command will not guess:

```
:modify.account fred programmer
Say what remote access FRED is to have: ssh, api, both or none
:modify.account fred programmer both
FRED may sign in over ssh and use the API
Account FRED is now PROGRAMMER
```

You cannot suspend your own account, or the one you are standing in. And a group account can be any tier except `administrator` — there is no single person behind it to put in a Windows group.

THE REMOTE ROUTES

The keyword says what the access is, not what to add. So

```
modify.account fred api
```

gives Fred the API **and takes ssh away**. If you want both, say `both`. The message afterwards always names both routes, so you can see what you have left him with.

REACHING THE OPERATING SYSTEM

`sh-on`, `sh-off`, `os-on` and `os-off` set the two fields of the person's `os.users` record without your having to edit it by hand — field 1 is the `sh` verb, field 2 is `OS.EXECUTE` and the two full-screen editors. See Administrator commands.

These four are switches, not names for one state, unlike the tier and the routes above — so `sh-off` leaves `OS.EXECUTE` alone.

WHAT REFUSES ON AN ADMINISTRATOR'S ACCOUNT

All three of ssh, the API and the operating system. An administrator has every route as a rule, and none of them is this verb's to change:

```
:modify.account don os-off
```

don is an administrator and always reaches the operating system

Downgrade the account first if that is really what you want. The tier is the only thing about an administrator that `modify.account` will change.

But the routes are local only, and that is not something you set. An administrator keeps ssh and the API for good, and both stop at this computer: a sign-in from anywhere else is refused, whatever the account record says.

An administrator may not sign in to this machine from another one.

Administration happens at the console, or through a remote desktop or remote-control product installed as a service. Ordinary and programmer accounts are not affected — their routes work from other machines exactly as before.

`set.password` **is now** `modify.password`. Same verb, same behaviour; the name changed because every account has a password from the moment it is made, so there is nothing to *set* for the first time.

Deleting an account

`delete.account` **asks once, then removes everything.** The single question names exactly what will go, including the Windows account when there is one to remove.

It will not delete a Windows account SD did not create. The question uses shorter wording in that case rather than promising something it will not do.

Two things that catch people out

1. `update.accounts` **never takes a verb away.** SD only ever *adds* records to a VOC at an update. An account created before a verb was withdrawn keeps it, and running `update.accounts` will not remove it.

`modify.account <tier>` **is the one that does remove them**, and it is the only thing that ever will. If you need a verb gone from an account, change the tier; `update.accounts` will not undo it afterwards either, because the tier is recorded and every update applies it.

2. `sdusers` **membership needs a fresh logon.** Same reason as grants. After being added to the group, sign out of Windows and back in, or you cannot read the data tree at all — and the symptom looks like a broken install rather than a permissions problem.